DATA Structure and Algorithms

(BCO 002B)

UNIT 5

Prepared By:
Ankur Raj
Assistant Professor
Department of Computer Science & Engineering
JECRC University

Syllabus

BCO002B	DATA STURCTURES AND ALGORITHM	4-0-0 [4]
UNIT 1	Introduction: Notions of data type, abstract data type and data structures. Importance of algorithms and data structures in programming. Notion of Complexity covering time complexity, space complexity, Worst case complexity & Average case complexity. BigOh Notation, Omega notation,Theta notation. Examples of simple algorithms and illustration of their complexity. Sorting- Bubble sort, selection sort, insertion sort, Quick sort; Heap sort; Merge sort; Analysis of the sorting methods. Selecting the top k elements. Lower bound on sorting.	
UNIT 2	Stack ADT, Infix Notation, Prefix Notation and Postfix Notation. Evaluation of Postfix Expression, conversion of Infix to Prefix and Postfix Iteration and Recursion- Problem solving using iteration and recursion with examples such as binary search, Fibonacci numbers, and Hanoi towers. Tradeoffs between iteration and recursion.	
UNIT 3	List ADT. Implementation of lists using arrays and pointers. Stack ADT. Queue ADT. Implementation of stacks and queues. Dictionaries, Hash tables: open tables and closed tables. Searching technique- Binary search and linear search, link list- single link list, double link list, Insertion and deletion in link list.	
UNIT 4	Binary Trees- Definition and traversals: preorder, post order, in order. Common types and properties of binary trees. Binary search trees: insertion and deletion in binary search tree worst case analysis and average case analysis. AVL trees. Priority Queues -Binary heaps: insert and delete min operations and analysis.	
UNIT 5	Graph: Basic definitions, Directed Graphs- Data structures for graph representation. Shortest path algorithms: Dijkstra (greedy algorithm) and Operations on graph, Worshall's algorithm , Depth first search and Breadth-first search. Directed acyclic graphs. Undirected Graphs, Minimal spanning trees and algorithms (Prims and Kruskal) and implementation. Application to the travelling salesman problem.	

Course Outcome	Program Outcome												Program Specifice Outcome		
	PO1	PO2	PO3	PO4	PO5	PO6	PO7	PO8	PO9	PO10	PO11	PO12	PSO1	PSO2	PSO3
CO1	H												M		
CO2			H		M								M		
CO3		H							M			L		H	
CO4	H	M											L		L
CO5		M		H											L

Course Objective

CO1:Show the understanding of asymptotic analysis of algorithm
CO2: Understand the concepts of Stacks and Queues with their application
CO3: Understand with utilization concepts of Linked List with their application
CO4:Use comprehensive knowledge Trees in problem solving.
CO5:Use comprehensive knowledge of Graph.

Books & References

Text Books:

1. Data Structures and Algorithms by Alfred V. Aho, Jeffrey D. Ullman and John E. Hopcroft , Addison-Wesley Series (1983)

Reference Books:

T.H. Cormen, C.E. Leiserson, and R.L. Rivest. Introduction to Algorithms.The MIT Press and McGraw-Hill Book Company, Cambridge, Massacusetts, 1990 (Available in Indian Edition).
Steven S. Skiena. The Algorithm Design Manual.Springer, Second Edition, 2008.
Data Structures and Algorithm Analysis in Java (3rd Edition) by Mark Allen Weiss, Addison Wesley(2011)

Graphs - Basic Introduction

- A Graph is a data structure that consists of a set of vertices (nodes) and edges.
- Used to represent networks such as roads, social connections, and web links.
- Types:
  - Directed Graph (edges have direction)
  - Undirected Graph (edges have no direction)
- Representations:
  - Adjacency Matrix
  - Adjacency List

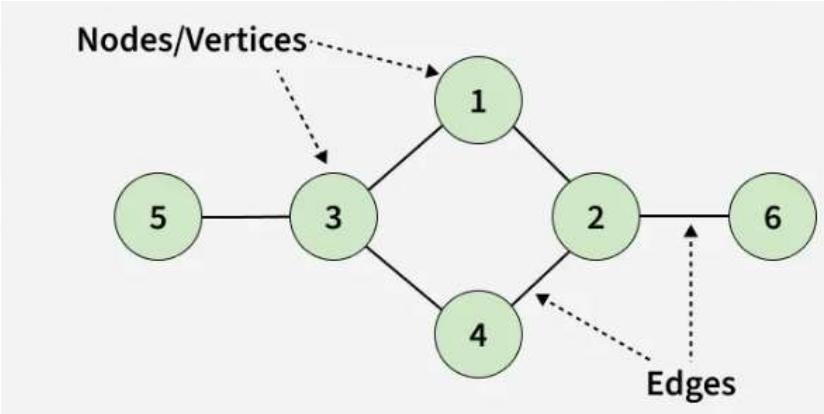
Components of Graph

JECRC UNIVERSITY
BUILD YOUR WORLD

Components of Graph Data Structure

•**Vertices:** Vertices are the fundamental units of the graph. Sometimes, vertices are also known as vertex or nodes. Every node/vertex can be labeled or unlabelled.

•**Edges:** Edges are drawn or used to connect two nodes of the graph. It can be ordered pair of nodes in a directed graph. Edges can connect any two nodes in any possible way. There are no rules. Sometimes, edges are also known as arcs. Every edge can be labelled/unlabelled



Common Terminology

- Vertex (Node): Fundamental unit representing data.
- Edge: Connection between two vertices.
- Degree: Number of edges connected to a vertex.
- Path: Sequence of vertices connected by edges.
- Cycle: A path that starts and ends at the same vertex.
- Connected Graph: Every vertex is reachable from every other vertex.

Depth First Search (DFS)

- DFS explores as far as possible along a branch before backtracking.
- Uses stack (LIFO) structure (can be implemented using recursion).

Algorithm:

1. Start from the source vertex.
2. Mark it visited.
3. Visit an adjacent unvisited vertex.
4. Repeat until all vertices are visited.

Applications: Topological sorting, cycle detection, path finding.

Breadth First Search (BFS)

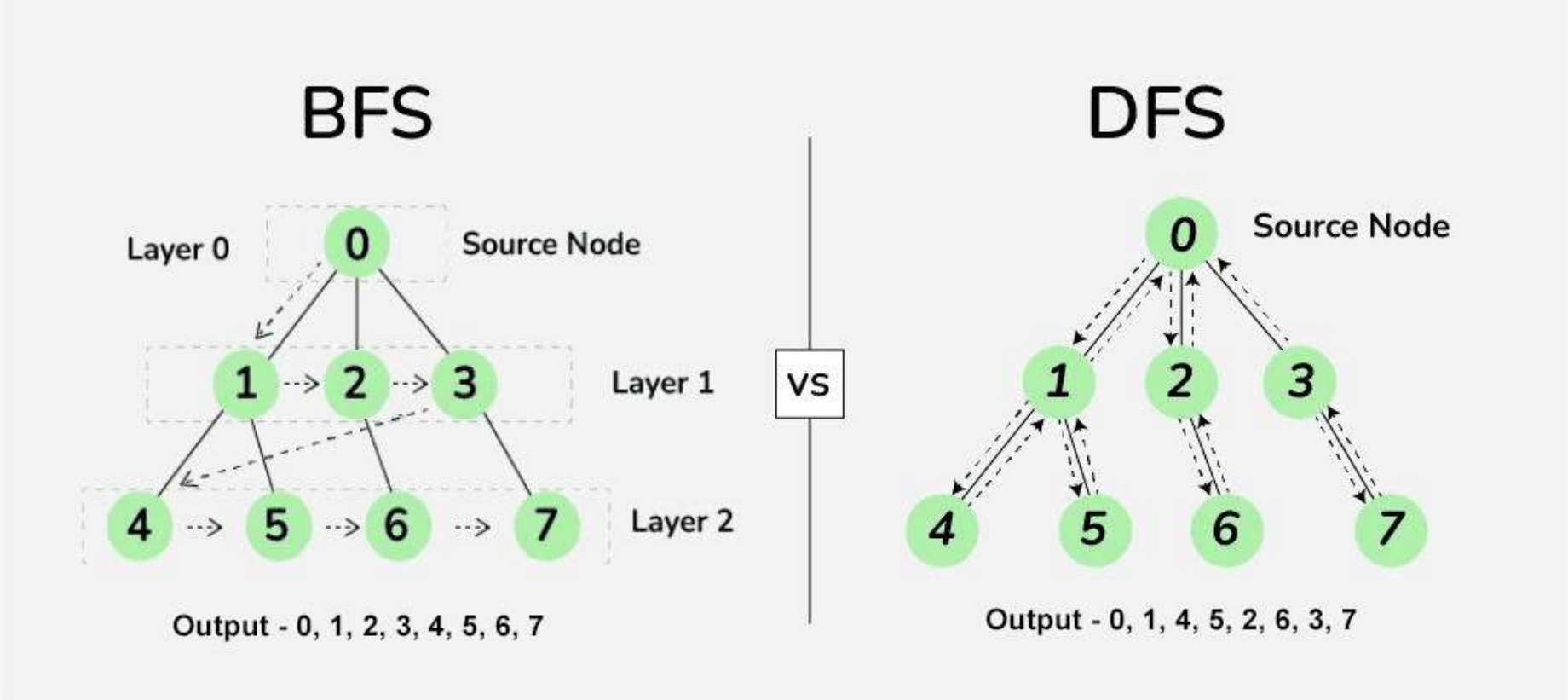
- BFS explores all neighbors before moving to the next level.
- Uses queue (FIFO) data structure.

Algorithm:

1. Start from the source vertex.
2. Visit and mark as visited.
3. Enqueue all adjacent unvisited vertices.
4. Repeat until queue is empty.

Applications: Shortest path in unweighted graphs, networking.

BFS Vs DFS


BFS
Layer 0
0
Source Node
1
2
3
Layer 1
4
5
6
7
Layer 2
Output - 0, 1, 2, 3, 4, 5, 6, 7
VS
DFS
0
Source Node
1
2
3
4
5
6
7
Output - 0, 1, 4, 5, 2, 6, 3, 7

Shortest Path Algorithm - Dijkstra

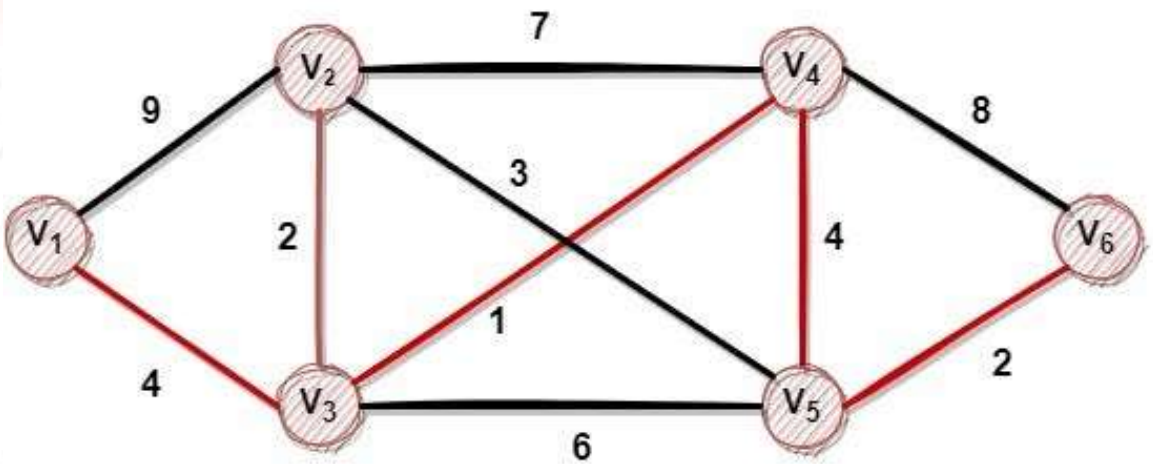
- Finds the shortest path from a source vertex to all other vertices.
- Works with non-negative edge weights.

Algorithm:

1. Assign distance=0 to source and ∞ to others.
2. Select vertex with minimum distance.
3. Update distances of adjacent vertices.
4. Repeat until all vertices are processed.

Applications: GPS, network routing.

Dijkstra's Algorithm



	Distance	Previous Node	Visited
V_1	0	None	True
V_2	6	V3	True
V_3	4	V_1	True
V_4	5	V3	True
V_5	9	V4	True
V6	11	V5	True

Path: V_1 - V_3 - V_4 - V_5 - V_6

Total Cost: 11

Warshall's Algorithm

The Warshall Algorithm, more commonly known as the Floyd-Warshall Algorithm, is a dynamic programming algorithm used to find the shortest paths between all pairs of vertices in a weighted graph. It can handle both positive and negative edge weights but cannot handle negative cycles (where the sum of edge weights in a cycle is negative).

Key Concepts:

•**All-Pairs Shortest Path:**

Unlike algorithms like Dijkstra's or Bellman-Ford, which find shortest paths from a single source, Floyd-Warshall computes the shortest path between every possible pair of vertices in the graph.

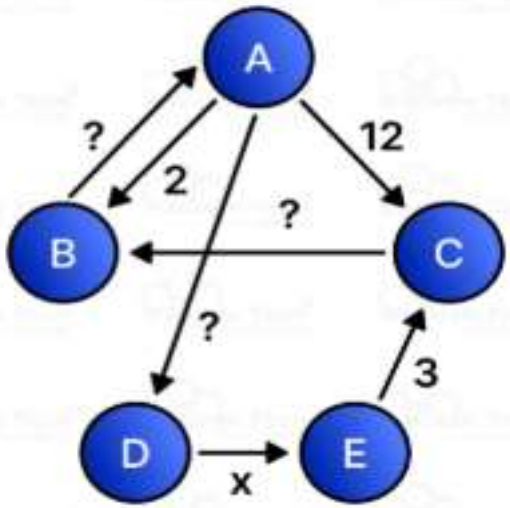
•**Dynamic Programming:**

The algorithm builds up the solution incrementally. It considers intermediate vertices one by one to find shorter paths.

•**Adjacency Matrix:**

The algorithm typically operates on an adjacency matrix representation of the graph, where `dist[i][j]` stores the shortest distance found so far between vertex `i` and vertex `j`. Initially, this matrix contains direct edge weights or infinity for non-existent edges.

Floyd Warshall Algorithm



	A	B	C	D	E
A	0	2	10	5	7
B	3	0	13	8	10
C	7	4	0	12	14
D	12	9	5	0	X
E	10	7	3	15	0

Algorithm:

```
For k = 1 to n:
  For i = 1 to n:
    For j = 1 to n:
      path[i][j] = path[i][j] OR (path[i][k] AND path[k][j])
```

Key Concepts in Floyd-Warshall Algorithm

•**Dynamic Programming:** This is a method used to solve problems by breaking them down into smaller, simpler subproblems. In the Floyd-Warshall algorithm, dynamic programming is used to update and keep track of the shortest paths between nodes by gradually improving the known distances as more paths are considered.

•**All-Pairs Shortest Paths:** The Floyd-Warshall algorithm finds the shortest paths for all pairs of nodes in the graph, not just from one node to all others. This means that after running the algorithm, you know the shortest path between any two nodes in the graph.

•**Matrix Representation:** The algorithm uses a matrix (a table with rows and columns) to represent the graph and store the distances between nodes. Each cell in the matrix represents the distance between two nodes. As the algorithm runs, it updates the matrix to reflect the shortest known distances between all pairs of nodes.

Minimum Spanning Tree (MST)

- A subset of edges that connects all vertices with minimum total weight.
- Does not form cycles.
- Common Algorithms: Kruskal's and Prim's Algorithm.

Applications: Network design, clustering, circuit design.

Kruskal's Algorithm

- Based on greedy approach.

Algorithm:

1. Sort all edges in non-decreasing order of weight.
2. Add edges one by one ensuring no cycles (use Disjoint Set).
3. Stop when there are (V-1) edges.

Applications: Building minimum cost networks.

Prim’s Algorithm

- Also based on greedy approach.

Algorithm:

1. Start with any vertex.
2. Select smallest edge connecting visited and unvisited vertex.
3. Repeat until all vertices are included.

Applications: Used in designing efficient networks like electricity grids.

Working of the Prim's Algorithm

Step 1: Determine an arbitrary vertex as the starting vertex of the MST. We pick 0 in the below diagram.

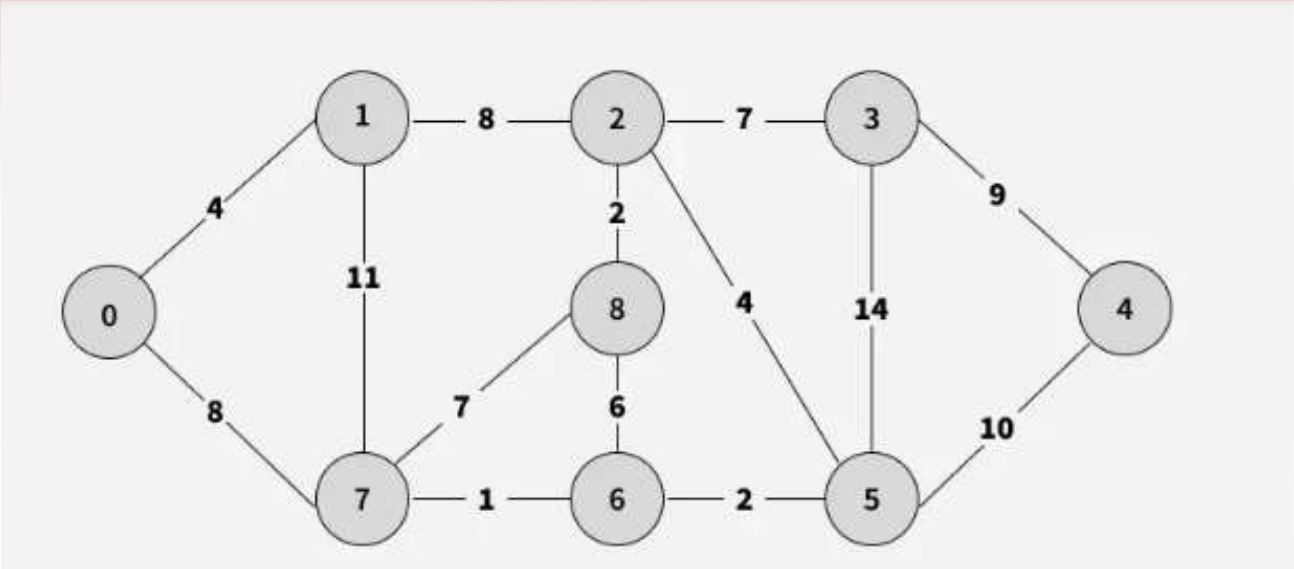
Step 2: Follow steps 3 to 5 till there are vertices that are not included in the MST (known as fringe vertex).

Step 3: Find edges connecting any tree vertex with the fringe vertices.

Step 4: Find the minimum among these edges.

Step 5: Add the chosen edge to the MST. Since we consider only the edges that connect fringe vertices with the rest, we never get a cycle.

Step 6: Return the MST and exit


0
1
2
3
4
5
6
7
8
4
8
8
7
9
2
11
4
14
7
6
10
1
2

Application: Travelling Salesman Problem (TSP)

- Objective: Find the shortest possible route visiting each city once and returning to the start.
- NP-hard problem.
- Approaches:
  - Brute Force: Check all permutations.
  - Dynamic Programming (Held-Karp Algorithm).
  - Approximation using MST or Genetic Algorithms.

Applications: Logistics, circuit design, route optimization.

Thank You